Arquitectura – Taller No.2 Jakarta EE (tier de negocio)

Módulo de gestión de solicitudes de recursos en emergencias sobre **Jakarta EE 10 + WildFly 37**, con el tier de negocio implementado como **EJB de sesión** y desacoplado del mecanismo de persistencia.

Documento	Qué muestra
01-componentes.md	Componentes, tiers y contenido del EAR
02-desacople-negocio-datos.md	Cómo se desacopló el negocio del tier de datos
03-secuencia-registrar.md	Flujo de registro, con el camino de rechazo
04-secuencia-atender.md	Flujo de atención y sus dos fallos de negocio
05-estados.md	Máquina de estados de la solicitud
06-modelo-datos.md	Tabla, unidad de persistencia y datasource

Los mismos documentos renderizados están en pdf/ y se regeneran con `./scripts/render-docs.sh`.

Stack

Aspecto	Decisión
Plataforma	Jakarta EE 10, Java 17
Servidor	WildFly 37.0.1 (contenedor de EJB, JTA, JPA, CDI, Jakarta REST)
Base de datos	H2 embebido, datasource `ResourceManagementDS`
Proveedor JPA	Hibernate (incluido en WildFly)
Empaquetado	EAR multi-módulo con Maven
Entrada al negocio	Fachada REST (vista `@Local`) y cliente Java por JNDI (vista `@Remote`)

Tipo de EJB

`@Stateless` para el bean de negocio y para el de datos.

- `@Stateful` mantendría estado conversacional por cliente. Cada operación aquí es autocontenida: recibe el comando o el identificador y termina. Añadiría afinidad de sesión y pasivación sin ninguna ganancia.
- `@Singleton` serializaría todas las invocaciones y convertiría el servicio en un cuello de botella. Además invita a guardar la colección de solicitudes dentro del propio bean, que es justamente el acoplamiento que el taller pide evitar.
- `@Stateless` deja que el contenedor atienda peticiones concurrentes desde su pool, sin sincronización manual, con el estado donde debe estar: en la base de datos.

Vistas del bean de sesión

Vista	Clientes	Paso de parámetros
`@Local` `ResourceRequestService`	La fachada REST, en la misma JVM	Por referencia
`@Remote` `ResourceRequestServiceRemote`	Cliente Java en otra JVM	Por valor (los DTO son `Serializable`)

Ambas extienden `ResourceRequestOperations`, así que el contrato se escribe una sola vez. El tier de datos solo publica vista local: no se expone fuera del servidor.